

1st-Spark AI — CTO Architecture & Strategy Document

Executive Summary

Spark AI is a commercial-grade, multi-platform AI SaaS platform designed for real users, real scale, and real revenue. This document covers every architectural, strategic, and technical decision you need to make before writing a single line of code. Every choice below is optimized for a **solo developer in 2026** who needs to move fast without accumulating technical debt that kills the product later.

1. Product Vision

Spark AI is a unified AI productivity platform that gives individuals, students, professionals, and businesses access to a suite of powerful AI tools — chat, document analysis, image understanding, research, coding assistance, and more — through a single, beautiful, subscription-based interface.

The vision is to be the **"Notion of AI tools"**: a platform where users don't need five different subscriptions to five different AI tools. Everything lives in one place, under one account, with one price.

The north star metric is **daily active usage** — users who return every day because Spark AI is embedded in their workflow, not just a tool they try once.

2. Long-Term Goals

Year 1: Establish product-market fit. Launch web app with core AI features. Acquire first 1,000 paying subscribers. Achieve \$10K MRR.

Year 2: Launch Android and iOS apps. Expand feature set. Build admin and analytics infrastructure. Reach \$50K MRR. Explore B2B/team plans.

Year 3: Enterprise features — SSO, team workspaces, API access, white-labeling. Target \$200K MRR. Explore Series A or profitability.

Long-term: Become a platform, not just an app — allow third-party integrations, a plugin ecosystem, and an API marketplace.

3. Target Users

Primary — Knowledge Workers & Professionals: Lawyers, consultants, analysts, marketers who need AI to accelerate research, writing, and document work. They pay for tools that save time.

Secondary — Students & Academics: University students and researchers who need PDF chat, study mode, and research assistance. High volume, lower ARPU, but strong virality.

Tertiary — Developers & Engineers: Coders who want a dedicated coding assistant with context, history, and project awareness. Willing to pay premium pricing.

Future — SMBs & Teams: Small businesses that want a shared AI workspace. This unlocks a B2B revenue stream with higher LTV and lower churn.

4. Core Features (MVP)

AI Chat: Multi-turn conversation with memory, model switching, and conversation history. The foundation everything else builds on.

PDF Chat: Upload a PDF, ask questions about it. Backend extracts, chunks, and embeds document content for RAG-based retrieval.

Image Analysis: Upload images for description, OCR, data extraction, or visual Q&A. Powered by vision models.

Web Search: AI answers grounded in real-time web results. Prevents hallucination on current events and gives users a research edge.

Coding Assistant: A dedicated mode with syntax-aware responses, code explanation, debugging, and code generation with diff views.

Conversation History: Full persistence of all conversations, searchable, organized by date and topic, with the ability to rename and delete.

Authentication: Email/password and OAuth (Google, GitHub). Secure session management.

User Dashboard: Overview of usage, active conversations, recent files, and subscription status.

Subscription System: Free tier with limits, Pro tier with full access, payment via Stripe.

Settings & Profile: API key management, theme, language, notification preferences, and account details.

5. Future Features (Post-MVP)

Voice Assistant: Real-time speech-to-text input and text-to-speech output. Hands-free AI interaction.

AI Image Generation: Text-to-image via Flux or DALL·E 3, with a gallery, prompt history, and download options.

Research Mode: Multi-step autonomous research agent — give it a topic, it searches, synthesizes, and produces a structured report.

Study Mode: Flashcard generation, quiz creation, and spaced repetition from uploaded study materials.

Team Workspaces: Shared conversations, shared document libraries, and role-based access for teams.

Admin Panel: User management, subscription oversight, usage analytics, model cost tracking, and content moderation tools.

Mobile Apps: Native Android and iOS apps with offline capability and push notifications.

Plugin/Integration Ecosystem: Zapier, Notion, Slack, Google Workspace integrations.

White-Label API: Let businesses embed Spark AI under their own brand.

6. Complete Technology Stack

Why this stack for a solo developer in 2026:

Every choice below optimizes for three things simultaneously: **speed of development**, **production readiness**, and **long-term scalability**. A solo developer cannot afford to rebuild the foundation later.

Frontend: Next.js 15 (App Router) with TypeScript. Next.js is the undisputed choice for AI SaaS in 2026 — it gives you SSR, SSG, API routes, streaming support, and a massive ecosystem. TypeScript is non-negotiable for a production system; it catches bugs before users do.

Styling: Tailwind CSS v4 + shadcn/ui. Tailwind gives you speed; shadcn gives you accessible, customizable components you own (not a dependency). Together they let one developer build professional UI fast.

State Management: Zustand for global client state (lightweight, no boilerplate), TanStack Query (React Query) for server state, caching, and background refetching.

Backend: Next.js API Routes + Node.js for simple endpoints, plus a dedicated Python FastAPI service for AI-heavy operations (embedding, RAG pipelines, agent workflows). Two-service architecture — not microservices complexity, but enough separation to scale AI workloads independently.

AI Orchestration: LangChain (Python) for RAG pipelines, agent workflows, and tool use. This is not a demo choice — LangChain in production with proper abstractions is powerful and maintainable.

Primary AI Models: Anthropic Claude API (claude-sonnet-4-6 for main chat, claude-haiku-4-5 for fast/cheap tasks). OpenAI GPT-4o as fallback and for vision. Pluggable model layer so you can swap or add models without rewriting core logic.

Vector Database: Pinecone (managed, scales automatically) for production, or pgvector (Postgres extension) for cost savings early on. Start with pgvector, migrate to Pinecone when scale demands it.

Primary Database: PostgreSQL via Supabase. Supabase gives you a managed Postgres instance, built-in Auth, Row Level Security, real-time subscriptions, and an auto-generated REST/GraphQL API. It dramatically reduces backend work for a solo developer without sacrificing power.

ORM: Prisma. Type-safe database queries, auto-generated types, migration management. The best ORM for TypeScript in 2026.

File Storage: Supabase Storage for user-uploaded files (PDFs, images). Falls back to AWS S3 for large-scale or when self-hosting. Supabase Storage is S3-compatible, so migration is trivial.

Authentication: Supabase Auth (email/password, OAuth providers, magic links, MFA). Handles sessions, JWTs, refresh tokens, and RLS integration out of the box. Zero auth infrastructure to maintain.

Payments: Stripe. Industry standard, best documentation, webhooks are reliable, and the React components (Stripe Elements) are production-ready.

Email: Resend (the modern replacement for SendGrid for developers). Simple API, React Email for templates, excellent deliverability.

Search: Built-in Postgres full-text search for conversation history initially. Upgrade to Typesense (self-hosted) or Algolia when needed.

Caching & Queues: Upstash Redis (serverless Redis) for rate limiting, session caching, and job queues. Upstash BullMQ for background job processing (embedding documents, sending emails, processing webhooks).

Real-time: Supabase Realtime for streaming conversation updates and live notifications.

Monitoring & Observability: Sentry for error tracking, PostHog for product analytics and feature flags, LangSmith for AI pipeline monitoring and prompt management.

Deployment: Vercel for the Next.js frontend and API routes (zero-config, edge network, preview deployments). Railway for the Python FastAPI backend (simple container deployment, scales easily). Both have generous free tiers and competitive pricing.

Mobile: React Native with Expo (Year 2). Shared business logic with the web app, fast development, and OTA updates.

CI/CD: GitHub Actions. Free, deeply integrated with GitHub, sufficient for solo developer workflows.

7. Frontend Architecture

The frontend is a Next.js 15 App Router application structured around feature-based organization, not layer-based organization. This means code that changes together lives together.

Routing structure: The App Router handles all routing. Public routes (landing page, pricing, blog) are statically generated. Authenticated routes (dashboard, chat, settings) are server-rendered with session validation. API routes handle lightweight backend operations.

Rendering strategy: Landing page and marketing pages use SSG for maximum performance and SEO. The dashboard and feature pages use SSR for auth-gated content. The chat interface uses client-side rendering with streaming — the AI response streams token by token using the Vercel AI SDK's `useChat` hook, which handles streaming, loading states, and error handling.

Component architecture: Three layers — UI primitives (shadcn/ui components), feature components (ChatMessage, PDFViewer, ConversationList), and page components (assembled from features). No feature component imports from another feature; they communicate through shared state.

Streaming: The Vercel AI SDK handles the complexity of streaming AI responses. Responses stream from the backend through a ReadableStream, and the UI updates in real time. This is critical for user experience — waiting 10 seconds for a full response feels broken; watching it type feels alive.

Responsive design: Mobile-first from day one. The same web codebase must work on phones because many users will access it on mobile before native apps exist.

8. Backend Architecture

The backend has two distinct services that serve different purposes.

Service 1 — Next.js API Routes (TypeScript): Handles user-facing API calls — auth callbacks, subscription management, conversation CRUD, file uploads, user settings, and webhook processing. These are thin, fast routes that mostly read/write the database and delegate heavy work to Service 2 or background queues.

Service 2 — Python FastAPI (AI Service): Handles all AI-heavy operations — RAG pipeline execution, document embedding, agent workflows, model orchestration, and vector search. This service is stateless and can be scaled horizontally when AI traffic grows. It communicates with Service 1 via internal HTTP calls or is called directly by the frontend for streaming responses.

Why two services: Python has the best AI/ML ecosystem in the world. LangChain, LlamaIndex, Sentence Transformers, and every cutting-edge AI library is Python-first. Trying to replicate this in Node.js means you're always playing catch-up. Keeping the AI logic in Python and the product logic in TypeScript gives you the best of both worlds.

Background jobs: Document processing (embedding a 100-page PDF) is not a synchronous operation. When a user uploads a PDF, the API accepts the upload, stores the file, enqueues a job, and returns immediately. The job worker picks up the task, processes the document, stores the embeddings, and updates the database with the result. The frontend polls or uses a real-time subscription to show progress.

API design: RESTful for all standard CRUD operations. Server-Sent Events (SSE) for streaming AI responses. WebSockets (via Supabase Realtime) for real-time notifications and status updates.

Rate limiting: Upstash Redis-backed rate limiting at the API route level. Different limits for free vs. Pro users, enforced before any expensive AI call is made.

9. Database Choice

Primary Database: PostgreSQL via Supabase

PostgreSQL is the right choice for 99% of SaaS applications. It handles relational data (users, subscriptions, conversations), JSON documents (flexible AI metadata), full-text search, and with

the pgvector extension, vector embeddings. It is battle-tested, massively scalable with read replicas and connection pooling, and Supabase makes operating it trivially easy.

Schema design philosophy: Core entities are Users, Conversations, Messages, Documents, Embeddings, Subscriptions, and UsageLogs. Row Level Security (RLS) at the database level ensures users can only access their own data — this is a security layer that cannot be bypassed even if application code has bugs.

Vector storage: pgvector handles vector embeddings directly in Postgres for the first stage. This eliminates an external dependency early. When you hit ~1 million vectors or need advanced approximate nearest neighbor search at scale, migrate to Pinecone. The migration is a data copy operation, not an architectural change.

Connection management: Supabase includes PgBouncer for connection pooling. The Python AI service uses asyncpg for async connections. The Next.js service uses Prisma with connection pooling configured for serverless environments.

10. Authentication Strategy

Provider: Supabase Auth

Supabase Auth is a production-grade authentication system built on top of PostgreSQL. It handles the full auth lifecycle — registration, login, session management, token refresh, OAuth, MFA, and password reset — without you maintaining any auth infrastructure.

Supported methods: Email/password, Google OAuth, GitHub OAuth, and Magic Link (passwordless email). GitHub OAuth is important for the developer/coder audience.

Session strategy: JWTs issued by Supabase, stored in httpOnly cookies (not localStorage — this prevents XSS token theft). Short-lived access tokens (1 hour) with automatic refresh via refresh tokens. The Next.js middleware validates sessions on every protected route before any page renders.

Row Level Security: Every database table has RLS policies that use the JWT's `user_id` claim to filter data. A user querying the database through Supabase's client library can never accidentally see another user's data — the database enforces it.

Admin authentication: Separate admin role stored in the users table. Admin routes in both the frontend and backend check for this role explicitly.

Future MFA: Supabase supports TOTP-based MFA. Enable it as a Pro feature or as an optional security setting.

11. AI Integration Strategy

Pluggable model layer: Never hardcode a single AI provider. Abstract all AI calls behind an interface — `generateChatResponse(model, messages, options)` — so you can swap Claude for GPT-4o, or add Gemini, without changing the calling code. This is critical for cost optimization and for hedging against provider downtime.

Model routing strategy:

- Claude Sonnet 4.6 → Primary chat, PDF analysis, research (best reasoning/cost balance)
- Claude Haiku 4.5 → Fast tasks, title generation, short summaries (10x cheaper)
- GPT-4o → Vision/image analysis, fallback when Claude is down
- Future: Gemini 2.0 Flash for speed-sensitive mobile use cases

RAG Pipeline (for PDF Chat):

1. User uploads PDF → stored in Supabase Storage
2. Background job extracts text using PyMuPDF
3. Text is chunked into 512-token segments with 50-token overlap
4. Each chunk is embedded using `text-embedding-3-small` (OpenAI) or Voyage AI embeddings
5. Embeddings stored in pgvector with metadata (page number, document ID, chunk index)
6. At query time: user question is embedded, top-K similar chunks retrieved via cosine similarity
7. Retrieved chunks + user question passed to Claude as context
8. Claude generates a grounded answer with citations

Streaming: All chat responses stream token by token. The Python service streams from the Claude API, and the Next.js backend proxies this stream to the frontend. The Vercel AI SDK's `StreamingTextResponse` handles the protocol.

Context management: Conversations maintain a sliding window of the last N messages plus a compressed summary of older messages. This prevents context window overflow while preserving conversation continuity.

Web search integration: Integrated directly as a tool in the LangChain agent. When a user asks a question that benefits from current information, the agent calls the Tavily Search API (the best LangChain-compatible search API in 2026), retrieves results, and synthesizes an answer with source citations.

Cost controls: Track token usage per user per month in the UsageLogs table. Free tier caps at a defined monthly token budget. When the budget is hit, requests are rejected with a friendly upgrade prompt. Pro tier has a generous but still enforced limit to prevent abuse.

LangSmith: All AI pipeline calls are traced through LangSmith. This gives you full observability — latency, token counts, prompt versions, error rates, and the ability to replay and debug production AI calls.

12. File Storage Strategy

Provider: Supabase Storage (primary), S3-compatible fallback

Supabase Storage is built on top of S3 and exposes an S3-compatible API. It integrates directly with Supabase Auth and RLS — you can write storage policies that ensure users can only access their own files. This is the correct choice for MVP and well into scale.

File handling:

- **PDFs:** Stored as-is. Size limit of 50MB per file on Pro, 10MB on Free. After storage, the background job pipeline triggers.
- **Images:** Stored as-is for analysis. Supabase Storage has built-in image transformation (resize, format conversion) via its CDN layer.
- **Generated images:** AI image generation outputs stored in a separate bucket with a public CDN URL.

Security: All user file buckets are private by default. Access requires a signed URL (time-limited) generated by the backend after verifying the requesting user owns the file. No file is ever publicly accessible without this check.

Cleanup: A background job runs daily to delete files associated with deleted conversations or accounts that have been closed, preventing storage cost accumulation.

Migration path: When you outgrow Supabase Storage or need multi-region storage, swap to AWS S3 directly. Because Supabase Storage is S3-compatible, the client code changes are minimal.

13. Payment Strategy

Provider: Stripe

Stripe is the only serious choice for a production SaaS in 2026. Their reliability, documentation, fraud tooling, and developer experience are unmatched.

Pricing model:

- **Free Tier:** Limited monthly messages, 1 document upload, no image generation, conversation history for 7 days
- **Pro Tier (\$15–\$20/month):** Unlimited messages (fair use), 50 document uploads/month, image generation, full conversation history, priority model access, web search
- **Future Team Tier (\$49–\$99/month):** Shared workspace, 5 seats, admin controls, team document library

Stripe products used:

- Stripe Subscriptions for recurring billing
- Stripe Customer Portal for self-service plan changes, cancellations, and payment method updates (this saves months of development)
- Stripe Webhooks for subscription lifecycle events (created, updated, canceled, payment failed) — these update your database in real time
- Stripe Checkout for the payment flow (hosted, PCI-compliant, no card data touches your servers)

Webhook handling: A dedicated Next.js API route receives and verifies Stripe webhook signatures, then updates the `subscriptions` table. Subscription status is checked on every authenticated request via a middleware layer — a canceled subscription immediately loses Pro access.

Usage-based future: If you add API access as a product, Stripe Meters (usage-based billing) can track and bill per-token usage. Stripe supports this natively.

14. Security Strategy

Security is not a feature you add later. These practices must be built in from day one.

Authentication security: httpOnly cookies prevent XSS token theft. CSRF protection on all state-changing routes. Short-lived JWTs with automatic rotation. MFA available for sensitive accounts.

Authorization: Every API endpoint explicitly checks user identity and ownership. Never trust client-supplied IDs — always verify ownership in the query (`WHERE user_id = authenticated_user_id`). Supabase RLS provides a second layer of enforcement at the database level.

Input validation: All inputs validated with Zod (TypeScript) on both the frontend and backend. AI prompts are not directly injected into system prompts without sanitization — prompt injection attempts are a real attack vector.

Rate limiting: Per-user, per-endpoint rate limiting using Upstash Redis. Aggressive limits on auth endpoints to prevent brute force. Gentler limits on chat endpoints, differentiated by subscription tier.

File upload security: File type validation on both client and server (check magic bytes, not just extension). File size limits enforced before processing. Malware scanning via an async VirusTotal API check for uploaded documents (async — don't block the user, but flag suspicious files).

API key security: If users are given API keys for platform access, keys are hashed (SHA-256) before storage. The plaintext key is shown to the user exactly once upon creation.

Secrets management: All secrets in environment variables, never in code. Vercel and Railway both have secure environment variable management. No secrets in `.env` files committed to version control — use `.env.example` templates.

Dependencies: Automated dependency scanning via GitHub Dependabot. Weekly dependency updates as a standing task.

Data encryption: Supabase encrypts data at rest. All traffic is HTTPS — no exceptions. Sensitive user data (payment info) never touches your servers — it goes directly through Stripe's SDK.

Compliance: GDPR compliance from day one — user data export, right to deletion, cookie consent, and a clear privacy policy. This is not optional if you have users in Europe.

Content moderation: Anthropic and OpenAI's APIs include content filtering. For user-generated content in community features (future), use a dedicated moderation endpoint.

15. Deployment Strategy

Frontend + Next.js API: Vercel

Vercel is built by the Next.js team. Zero-configuration deployment, global edge network, automatic HTTPS, preview deployments for every pull request, and serverless function scaling. For a solo developer, the operational overhead is near zero.

Python AI Service: Railway

Railway is the best platform for deploying Dockerized Python services in 2026 for developers who don't want to manage Kubernetes. Define a Dockerfile, connect your GitHub repo, and Railway handles the rest — auto-deploys on push, environment variables, private networking, scaling controls, and built-in metrics.

Database: Supabase (managed)

Supabase handles all database operations — backups, failover, connection pooling, monitoring. You never SSH into a database server.

CDN: Vercel's built-in CDN handles all static assets. Supabase Storage's CDN handles user files. No separate CDN configuration needed.

Environment management: Three environments — **development** (local), **staging** (Vercel preview + Railway staging service), **production**. Database migrations are run against staging first, then production.

Deployment flow: Push to **main** → GitHub Actions runs tests → Vercel auto-deploys frontend → Railway auto-deploys backend → Sentry releases tagged → deployment complete. Total time under 5 minutes.

Monitoring: Sentry catches errors in real time with stack traces and user context. PostHog tracks user behavior and feature usage. Upstash dashboard shows Redis usage. Railway dashboard shows backend metrics. Supabase dashboard shows database performance.

Backup strategy: Supabase takes daily automated backups with point-in-time recovery on paid plans. User files in Supabase Storage are durable (backed by S3). Additional exports of critical tables run weekly via a GitHub Actions cron job.

16. Mobile App Strategy

Framework: React Native with Expo (Year 2)

React Native with Expo is the right choice for a solo developer building cross-platform mobile in 2026. You write one codebase and ship to both Android and iOS.

Why not Flutter: Your entire team (you) knows JavaScript/TypeScript. React Native lets you share types, API clients, and business logic with the web app. Context switching between Dart and TypeScript is a productivity killer for a solo developer.

Why not a PWA: A Progressive Web App is a viable bridge for Year 1, but it lacks push notifications, deep OS integration, and the App Store presence that drives discovery and trust. Native apps are worth the investment in Year 2.

Code sharing strategy: Create a shared **packages/core** package in a monorepo (Turborepo) containing API client functions, TypeScript types, validation schemas, and utility functions. The web app and mobile app both import from this package. UI components are platform-specific (Next.js components for web, React Native components for mobile), but everything underneath is shared.

Expo advantages: Over-the-air (OTA) updates let you push bug fixes and non-native changes to users without waiting for App Store review. Expo handles the build and submission pipeline for both stores. Expo Go lets you test on real devices instantly during development.

Mobile-specific features: Push notifications for conversation replies and feature announcements. Biometric authentication (Face ID, fingerprint) for app access. Native voice recording for the Voice Assistant feature. Offline mode for reading conversation history without connectivity.

App Store strategy: Launch Android first (faster review times, easier side-loading for beta testing). iOS follows within weeks. Use Expo's EAS (Expo Application Services) for managed builds and submissions.

17. Scalability Strategy

The architecture described above can scale from 0 to 100,000 users without fundamental changes. Here is how each layer scales.

Frontend: Vercel's edge network automatically scales. Static pages are served from CDN globally. Serverless API routes scale horizontally per request. No action required.

Python AI Service: Deploy on Railway with auto-scaling configured. The service is stateless, so horizontal scaling is a slider adjustment. At very high volume, move to AWS ECS with Fargate for more fine-grained control.

Database: Supabase's managed Postgres handles significant load. When read load grows, enable read replicas in Supabase — read-heavy queries (fetching conversation history) route to replicas. PgBouncer handles connection pooling automatically. At extreme scale, partition the `messages` table by `user_id` range.

Vector storage: Migrate from pgvector to Pinecone when you exceed ~2 million vectors or need sub-10ms retrieval at scale. Pinecone is a managed service — the migration is a data operation, not an architecture change.

Caching: Upstash Redis caches expensive repeated queries (user subscription status, model configurations, rate limit counters). Cache hit rate target is above 80% for these hot paths.

Background jobs: Upstash QStash or BullMQ with Redis for job queues. Add more workers (Railway instances) when job processing latency grows. Jobs are idempotent — safe to retry on failure.

Cost scaling: The primary cost driver is AI API usage. As you scale, implement smarter caching (cache identical or semantically similar queries), use cheaper models for appropriate

tasks, and negotiate volume pricing with Anthropic and OpenAI. At \$50K MRR, enterprise agreements become available.

Geographic scaling: Vercel edge functions run in 40+ regions globally. For the Python service, deploy Railway instances in multiple regions behind a global load balancer when user distribution demands it.

18. Development Roadmap

Phase 0 — Foundation (Weeks 1–3)

Set up the monorepo with Turborepo. Configure Next.js 15 with TypeScript, Tailwind, and shadcn/ui. Set up Supabase project with initial schema and RLS policies. Configure Prisma with the database. Set up Supabase Auth with email/password and Google OAuth. Deploy a skeleton to Vercel. Set up the Python FastAPI service on Railway with a health check endpoint. Configure GitHub Actions for CI. Set up Sentry and PostHog. This phase ends with a deployed, authenticated, empty shell — no AI features yet. This is the hardest phase to be disciplined about, but skipping it creates chaos later.

Phase 1 — Core AI Chat (Weeks 4–7)

Build the conversation data model and API. Implement the chat interface with streaming responses using the Vercel AI SDK. Integrate Claude API through the Python service. Build conversation history — list, open, rename, delete. Add model selection (Sonnet vs Haiku). Add the basic user dashboard. By the end of this phase, you have a working AI chat product. This is your first deployable feature.

Phase 2 — PDF Chat & Document Features (Weeks 8–11)

Build the file upload UI and Supabase Storage integration. Implement the Python-based PDF processing pipeline — extraction, chunking, embedding with pgvector. Build the RAG query pipeline. Create the PDF Chat interface with source citation display. Add document library management. Add background job processing with progress indicators. This phase unlocks a major differentiating feature.

Phase 3 — Subscriptions & Monetization (Weeks 12–14)

Integrate Stripe — checkout, webhooks, customer portal. Build the pricing page. Implement usage tracking and tier enforcement (rate limiting, feature gates). Build the subscription management UI in settings. Set up Stripe test mode end-to-end, then flip to live. This phase turns the product into a business.

Phase 4 — Web Search, Image Analysis & Additional Modes (Weeks 15–19)

Integrate Tavily for web search — build the search tool into the LangChain agent. Build the image upload and analysis interface using GPT-4o vision. Create the Coding Assistant mode with syntax highlighting and code-specific UI. Build the Research Mode with multi-step agent workflow. Build the Study Mode with flashcard generation. Each of these is a distinct mode in the UI with specialized prompting and display.

Phase 5 — Admin Panel & Operations (Weeks 20–22)

Build the internal admin dashboard — user management, subscription overview, usage analytics, model cost tracking. Add the ability to ban users, issue refunds, and adjust subscription tiers manually. Set up LangSmith for AI pipeline monitoring. Add PostHog dashboards for product metrics. This phase makes the business operable.

Phase 6 — Polish, Performance & Launch (Weeks 23–26)

Comprehensive cross-browser and mobile web testing. Performance audit — Lighthouse scores, Core Web Vitals, API response times. Security audit — penetration testing basics, dependency audit, secret scanning. Onboarding flow for new users. Email sequences via Resend — welcome email, feature discovery, upgrade prompts. Landing page and pricing page SEO optimization. Public beta launch.

Phase 7 — Voice Assistant & Image Generation (Months 7–8)

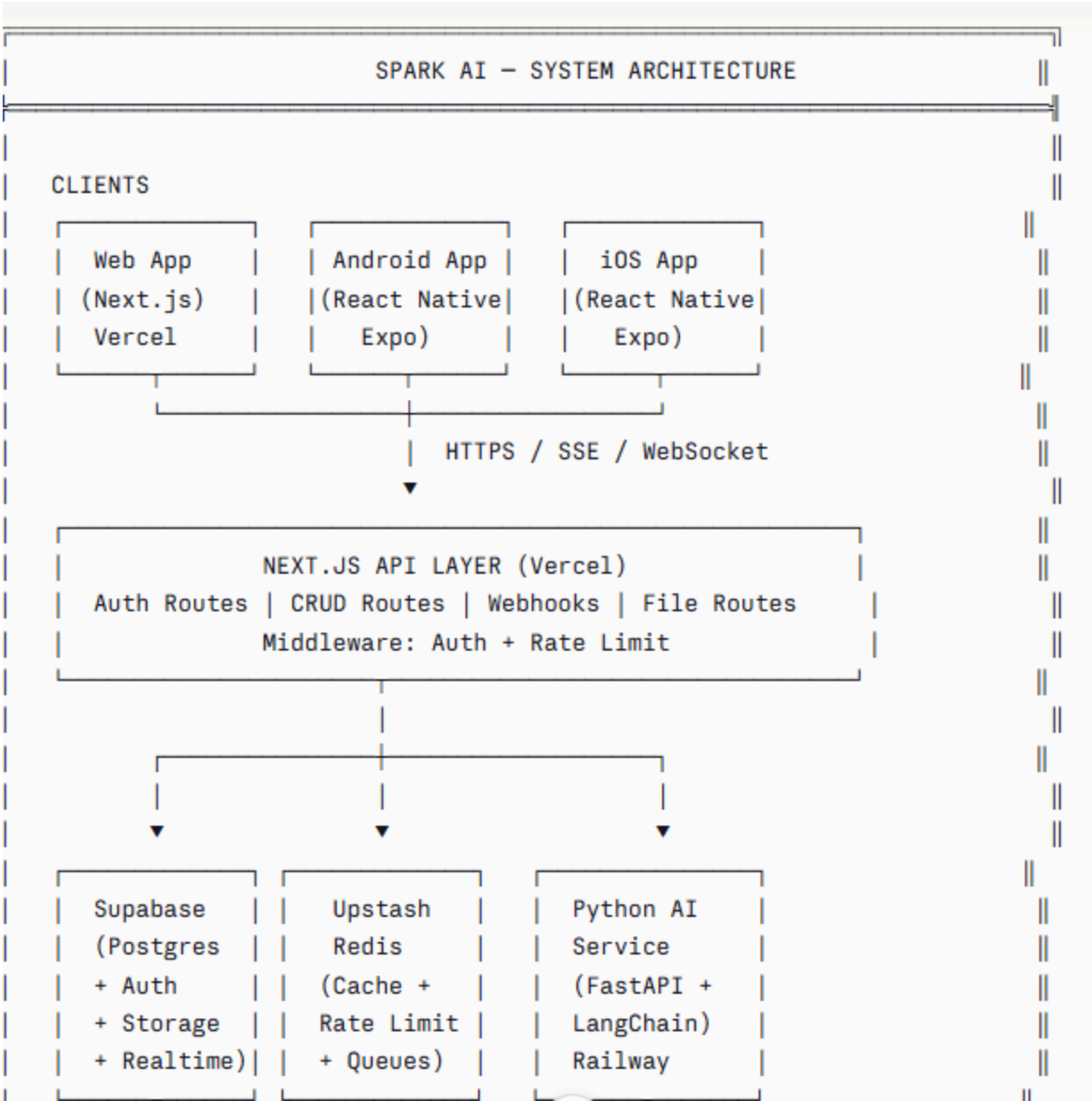
Integrate Whisper API for speech-to-text. Integrate ElevenLabs or OpenAI TTS for voice output. Build the voice interface. Integrate DALL·E 3 or Flux for image generation. Build the image generation UI with prompt history and gallery.

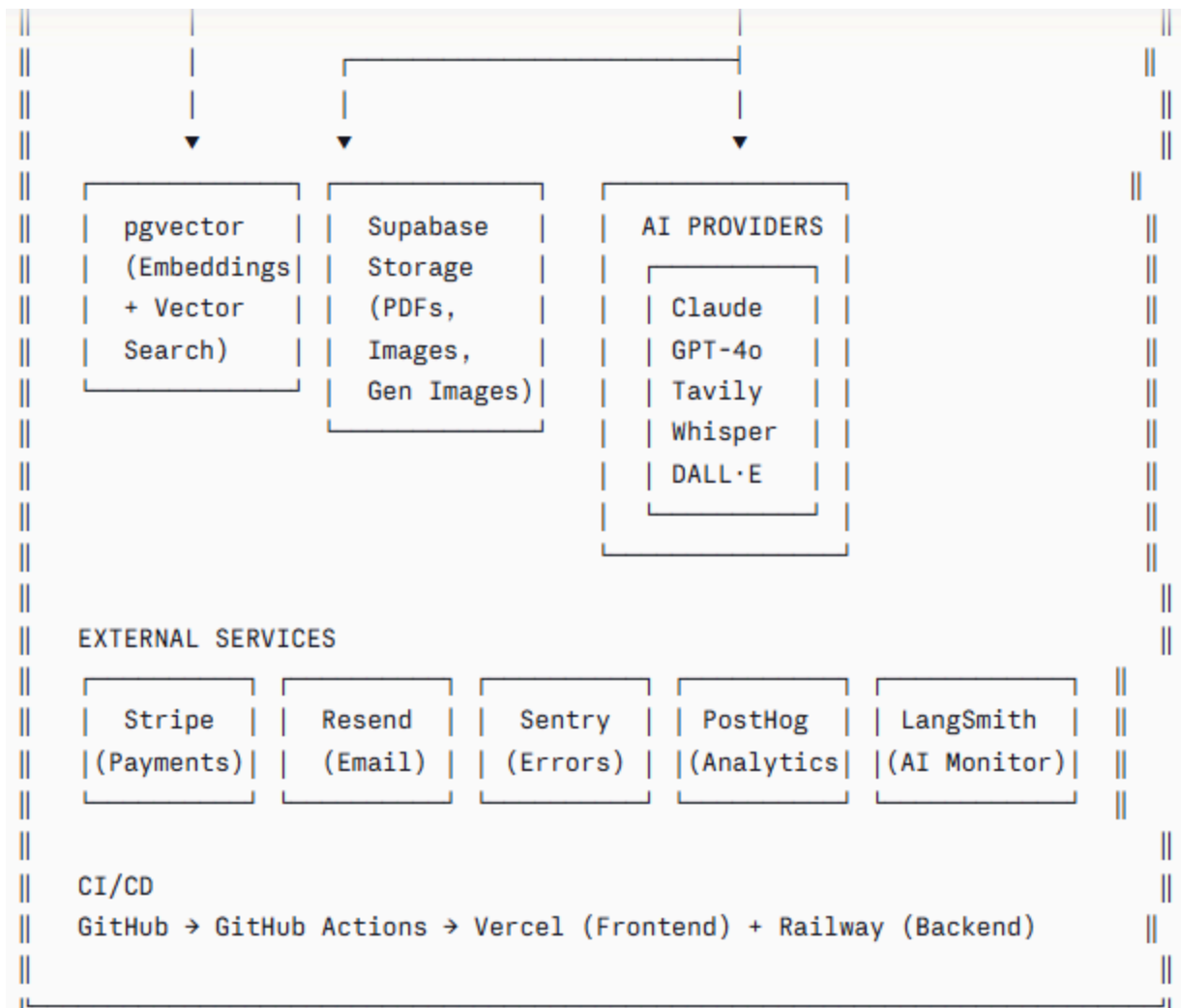
Phase 8 — Mobile Apps (Months 9–12)

Set up React Native with Expo in the monorepo. Build the mobile-optimized chat interface. Implement push notifications. Add biometric auth. Build and submit Android app to Play Store. Build and submit iOS app to App Store. Implement OTA update pipeline.

Phase 9 — Team Features & Enterprise (Year 2)

Workspace concept — invite team members, shared conversations, shared document library. Role-based access control (admin, member, viewer). Team billing — one subscription, multiple seats. SSO integration for enterprise. API access for developers. This phase opens the B2B revenue channel.





DATA FLOW – PDF Chat Example:

User uploads PDF

→ Next is API stores file in Supabase Storage

→ Enqueues job in Upstash Redis

- Returns "processing" status to client
- Background worker picks up job
- Python service extracts + chunks + embeds PDF
- Embeddings stored in pgvector
- Status updated in Postgres → Realtime notifies client

ser asks a question

- Next.js API routes to Python AI Service
- Embeds question → vector search in pgvector
- Top chunks retrieved → passed to Claude as context
- Claude streams answer → SSE back to client
- Message saved to Postgres conversation

KEY DECISIONS SUMMARY:

Layer	Technology
Frontend	Next.js 15 + TypeScript + Tailwind
UI Components	shadcn/ui
State	Zustand + TanStack Query
Backend API	Next.js API Routes
AI Service	Python FastAPI + LangChain
Primary Database	PostgreSQL via Supabase
Vector Database	pgvector → Pinecone (at scale)
Authentication	Supabase Auth
File Storage	Supabase Storage → S3
Caching/Queues	Upstash Redis
Payments	Stripe
Email	Resend
Primary AI	Anthropic Claude
Web Search	Tavily API
Frontend Deploy	Vercel
Backend Deploy	Railway
Mobile	React Native + Expo (Year 2)
Monitoring	Sentry + PostHog + LangSmith
CI/CD	GitHub Actions

Final CTO Notes

This architecture is designed to take you from zero to \$100K MRR without a rewrite. Every technology choice is justified by three criteria: does it reduce solo-developer operational burden, does it scale without architectural changes, and does it have a clear migration path when you outgrow it.

The biggest risks for this project are scope creep (building Phase 5 features during Phase 1) and premature optimization (over-engineering the infrastructure before you have users). The roadmap is deliberately sequenced to get you to a deployable, monetizable product as fast as possible while laying a foundation that doesn't collapse under growth.

Awaiting your approval to proceed to Phase 0 implementation.